

Guardrails

A hands-on course in building agents by building their harness

Lesson 6

The box limits reach, not whether

Inside Lesson 5's box, the agent still runs whatever the model asks, as soon as it asks, for as long as it keeps asking.

- `rm -rf` on the mounted folder: your real project, gone
- Stuck retrying one command: it goes on until you notice the bill

The model proposes; the harness decides

Guardrails are rules about what the harness is allowed to do, checked before it does it: which commands may run, when a person gets a say, when a run has gone on long enough.

Built on control flow, which decides whether the next step happens at all:

- It turns a tool request into a command being run
- It turns the result into another model call
- A guardrail is a decision at one of those two points, so it lives there

A few checks, where the loop already passes

- **A gate before each tool:** *allow* it, *ask* a person first, or *deny* it
- **A refusal the model can read:** in the result's slot, marked as an error, saying why
- **Limits before each call:** count the steps and tokens; past the limit, stop
- **An interrupt:** ESC stops it at once, thinking, saying or acting, tells the model, and hands the run back

The concept: a gate, and nothing else

`guardrails.py` takes commands on the command line and decides about each one (abridged):

```
DENY = {r"\bsudo\b": "no sudo", r"rm\s+~\w*[rf]": "no recursive or forced deletes", ...}
SAFE = {"ls", "cat", "head", "tail", "wc", "grep", "pwd", "date", "echo"}

def safe(cmd):
    return not re.search(r"[;<>$`\n()]", cmd) and all(...) in SAFE ...

def gate(cmd):
    for pattern, why in DENY.items():
        if re.search(pattern, cmd): return f"denied: {why}"
    if safe(cmd): return None
    try: answer = read("allow it? [y/N] ")
    except EOFError: answer = ""
    return None if answer.strip().lower() == "y" else "the person said no"
```

Deny on a pattern; **allow** what's only safe programs; **ask** about the rest

Six commands, two questions

I answered `y`, then `n`:

```
$ wc -l notes.txt
3 notes.txt
$ cat notes.txt | head -2
buy milk
call Sam
$ find . -name "*.txt"
allow it? [y/N] y
./notes.txt
$ rm a.log
allow it? [y/N] n
[the person said no]
$ rm -rf .
[denied: no recursive or forced deletes]
$ cat .env
[denied: secrets are off limits]
```

With Jev: one question, after the list

`jev_guardrails.py` is `guardrails.py` plus only these lines (abridged): `kind()`, and three in `gate()`, after the list

```
def kind(cmd):  
    try:  
        answer = jev.system_one({"command": cmd}, {"kind": KIND}).choices["kind"]  
        return answer.choice, answer.confidence  
    except Exception:  
        return "no answer", 0.0  
...  
choice, confidence = kind(cmd)  
print(f"[Jev: {choice}, {confidence:.2f}]" )  
if choice == "read" and confidence >= SURE: return None
```

- `KIND`: `read`, `write`, `delete` or `other`; a sure read runs, the rest is asked
- Asking is **model interface**; running or asking is **control flow**

The questions the list would have asked

I answered `y`, then `n` (shortened):

```
$ find . -name "*.txt"
[Jev: read, 1.00]
./notes.txt
$ ls; cat notes.txt
[Jev: read, 1.00]
...
$ touch draft.txt
[Jev: write, 1.00]
allow it? [y/N] y
$ rm a.log
[Jev: delete, 1.00]
allow it? [y/N] n
[the person said no]
```

quark's policy and gate

Lesson 5's `quark.py` plus 66 lines, 334 in all. At the top of `# — control flow —` (abridged):

```
MAX_STEPS, MAX_TOKENS = 20, 200_000
SAFE = {"ls", "cat", "head", "tail", "wc", "grep", "pwd", "date", "echo", "du", ...}
DENY = re.compile(r"\bsudo\b|rm\s+~\w*[rf]|mkfs|git\s+push|(curl|wget).*\|s*(ba)?sh|\s*.env\b")
KIND = Choice(instructions="What does the shell command in `command` do? ...", criteria={...})
def guard(cmd):
    if DENY.search(cmd): return "blocked by policy"
    if not re.search(r"[;<>$`\n()]", cmd) and all(... in SAFE ...): return None
    kind = ask({"command": cmd}, KIND)
    if kind and kind["choice"] == "read" and kind["confidence"] >= SURE: return None
    why = f" (Jev: {kind['choice']}, {kind['confidence']:.2f})" if kind else ""
    answer = read(f"allow `{cmd}`?{why} [y/N] ")
    return None if answer.lower() == "y" else "the person said no" + ...
```

`guard()`: deny, safe list, Jev, or ask

It returns `None` if the command may run, or the reason it may not:

1. Matches `DENY`: refused outright, and no one is asked
2. Only safe programs: no `; & < > $`, backtick, newline or `(`, every pipe piece checked
3. Otherwise Jev, through Lesson 5's `ask()`: a sure `read` runs
4. Anything else goes to the person, with Jev's answer as the reason
5. Anything but `y` is a no, and what they typed goes back to the model

Limits, checked before every call

At the top of the loop, and where each call is counted (abridged):

```
if steps >= MAX_STEPS or spent >= MAX_TOKENS:
    print(f"[stopped: {steps} steps, {spent} tokens]")
    if not chat or (input := read("\n> ")) == "/q": break
    add(working_memory, {"role": "user", "content": input})
    steps, spent = 0, 0
    continue
...
with listening():
    response = call(unless_esc, ..., messages=working_memory)
    output = response.content
    steps += 1
    spent += response.usage.input_tokens + response.usage.output_tokens + ...
```

In a one-shot run, a limit is the end. In a chat it hands back to you, and the counts start again after each new input.

A refusal goes where the result would have

In the output loop, between printing the command and running it in the box (abridged):

```
print(f"$ {block.input['cmd']}")
if ESC.is_set():
    input.append(..., "content": "[your doing never reached the world]")
    continue
if (no := guard(block.input["cmd"])):
    print(f"[{no}]")
    input.append({"type": "tool_result", ..., "content": no, "is_error": True})
    continue
```

- `continue` skips the run: the command never starts, and the model's next turn still makes sense
- The `ESC` check above the gate: once you've pressed ESC, nothing else starts

The interrupt: listening for ESC

ESC is input from a person, so in `# — input —`, after `read()` (abridged):

```
ESC = threading.Event()
SAYING = "[other self interrupted what you were saying – acknowledge]"
DOING = "[other self interrupted what you were doing – acknowledge]"
def watch(stop):
    ...
@contextlib.contextmanager
def listening():
    ESC.clear()
    if not sys.stdin.isatty(): yield; return
    ...
```

- `watch()` reads one key at a time in a thread; an ESC on its own sets `ESC`
- quark listens only while it thinks, says or acts, never while `read()` waits for you

Stopping what it says: the next piece ends it

One line in `call()`, `unless_esc` in place of `show`, and a check right after the call (abridged: end-of-line comments cut):

```
        for event in stream:
            if each(event): return stream.current_message_snapshot
def unless_esc(event):
    if ESC.is_set(): return True
    show(event)
...
    if response.stop_reason is None:
        print()
        add(working_memory, {"role": "user", "content": SAYING})
        continue
```

Stopping what it does: `kill -9 -1` in the box

The command now starts with `Popen`, and quark waits a tenth of a second at a time (abridged):

```
with listening():
    doing = subprocess.Popen(["docker", "exec", box, ..., block.input["cmd"]], ...)
    while True:
        try: done = ...(doing.communicate(timeout=0.1)[0]); break
        except subprocess.TimeoutExpired:
            if ESC.is_set(): subprocess.run([..., "kill -9 -1"], ...)
    ...
    if ESC.is_set(): done.stdout = "[your doing stopped before done]"
```

- Every command in the box stops; the box itself survives
- What it had printed is dropped; the model gets `DOING`

Safe runs; a delete waits for you

`wc -l notes.txt` is on the safe list, so it just ran. Then: delete the `.log` files. I typed `y` (shortened):

```
$ ls -la *.log; ls -A
-rw-r--r-- 1 root root 4 Oct  6 23:18 a.log
-rw-r--r-- 1 root root 4 Oct  6 23:18 b.log
...
$ rm a.log b.log; ls -A
allow `rm a.log b.log; ls -A`? (Jev: delete, 1.00) [y/N] y
.env
.quark
notes.txt
```

A no with a reason; a deny that asks no one

I answered `no`, put it in a folder called `drafts`, then `y` (shortened):

```
$ touch notes2.txt && ls -l notes2.txt  
[the person said no: no, put it in a folder called drafts]  
$ mkdir -p drafts && touch drafts/notes2.txt && ls -l drafts/notes2.txt
```

Then: print `.env`, then `rm -rf .` (shortened):

```
$ cat .env  
[blocked by policy]
```

A read off the list; a limit

`find` isn't in `SAFE`, nothing was on stdin, and nothing was asked:

```
$ find . -name "*.txt" -type f
./drafts/notes2.txt
./notes.txt
```

With no Jev, the same read was put to the person, and got a no. With `MAX_STEPS` lowered to 2 in a copy (shortened):

```
$ pwd
/tmp/d06/q

[stopped: 2 steps, 16953 tokens]
```

ESC while it acts: the count stops partway

Count to ten, one a second; ESC four and a half seconds after the command appeared (shortened):

```
$ for i in 1 2 3 4 5 6 7 8 9 10; do echo $i; sleep 1; done  
[your doing stopped before done]
```

You interrupted the count, so I stopped it. The command didn't finish, and I don't know how far it got because the output was cut off.

- It ran without asking: Jev was sure it only prints

ESC while it says: cut off mid-word

ESC a second after its first full stop (shortened):

```
> Tell me the fable of the tortoise and the hare in about 400 words  
Here is the fable of the tortoise and the hare.  
...  
The hare burst out laughing. "You
```

I see you interrupted me, so I'll stop there. I hadn't finished the fable, which was meant to run about 400 words. Do you want me to start again from the beginning, pick up where I left off, ...

- It went on to offer to "pick up where I left off", and it can't
- None of what it said was kept: it knows it was cut off, not where

ESC while it thinks: back to the request

ESC 0.6 seconds after asking, before it said a word. The blank line is the whole stopped response (shortened):

```
> Make a file called hello.txt containing the word hello.  
  
$ echo hello > hello.txt && cat hello.txt  
allow `echo hello > hello.txt && cat hello.txt`? (Jev: delete, 0.85) [y/N] /q  
[the person said no]
```

- Jev: a delete, at 0.85, because `>` replaces what was there
- Nothing reached the folder: there's no `hello.txt`
- Keeping what was cut short is Lesson 8's job

Other things we could do

- **How a command is judged:** text patterns, a shell parser, per-tool rules, or a second model
- **What the answers are:** allow once, for the session, always, or with these arguments
- **Who gets asked:** the terminal, someone on Slack or Telegram, a second agent, or nobody
- **What counts as too much:** steps, tokens, dollars, wall-clock time, or repeating
- **How a run is interrupted:** a key, a message mid-run, a kill switch

As a product alone: **Open Policy Agent, NeMo Guardrails, Guardrails AI.**

The rule: decide before the harness does it

The rule: check each tool request where control flow turns it into a command, and the run where it goes around again. Let a second model settle what a list can't, only to spare the person a question. Say no in a way the model can read.

What guardrails never do, from inside control flow:

- **Model interface:** one line, so a response can be stopped; Jev through `ask()`
- **Output:** an allowed command runs in the box exactly as before
- **Context:** a refusal is an ordinary tool result in the result's slot
- **Input:** Lesson 2's `read()`, plus a watcher that hears ESC while quark works

What's missing: you can stop a run, not see one

- How long did each step take?
- How many tokens, and what did that cost?
- Which command did the guard refuse, and which did the box kill?
- The episode has every message, but it's written for the model: no timings, no token counts, no exit codes

Next: Observability

Lesson 7